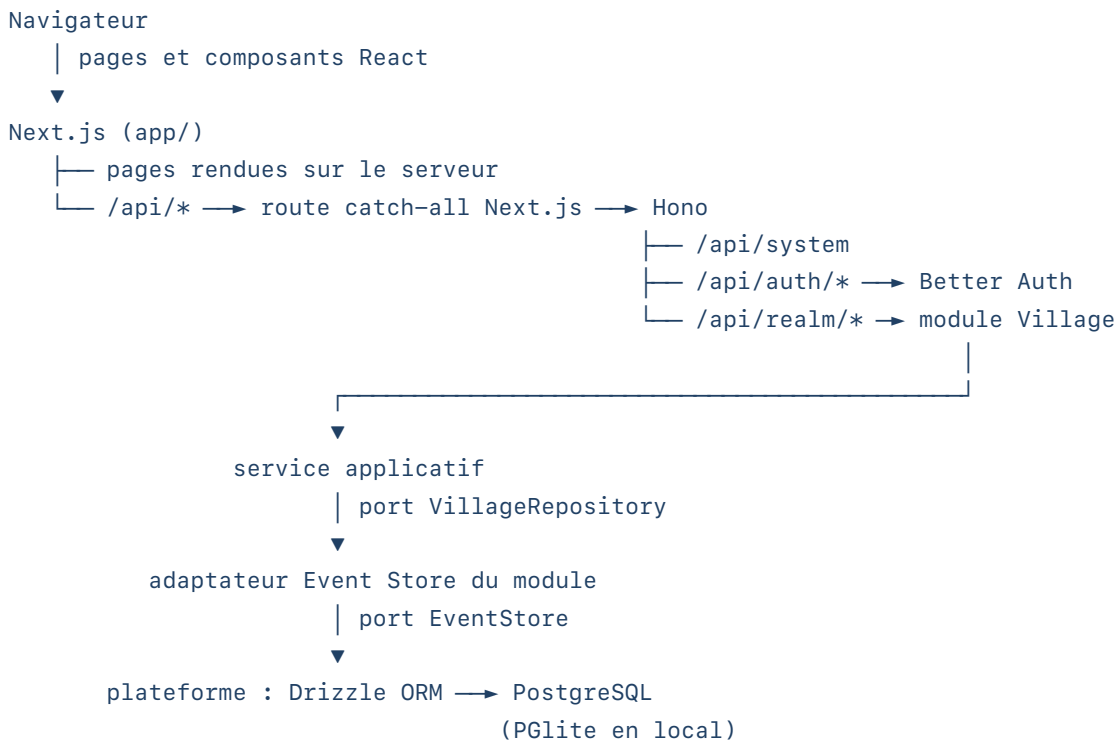
Royaumes — comprendre l'architecture de l'application

Guide technique — 19 octobre 2026

Ce guide décrit l'application telle qu'elle existe dans le dépôt : ses couches, le parcours d'une requête, le module Village, la persistance, l'authentification, les dépendances et les commandes utiles. Les modules Donjon, Classement, Marché et Taverne apparaissent dans les intentions d'architecture, mais ne sont pas encore implémentés.

1. Vue d'ensemble

Royaumes est un jeu web construit comme un monolithe modulaire. Il n'y a qu'une application Next.js, un serveur API Hono et une base de données. Le code est cependant séparé en modules métier et en plateforme technique, avec des règles automatiques qui empêchent les dépendances de traverser leurs frontières.



Technologies principales : Next.js 16 et React 19 pour l'application web, TypeScript 6, Hono 4 pour l'API, Better Auth 1.7 pour l'identité, Drizzle ORM 0.45 pour l'accès aux données, et PostgreSQL (PGlite en local) pour la persistance.

2. Où se trouve le code ?

`app/`	Pages Next.js et point d'entrée HTTP de l'API
`page.tsx`	Redirection d'accueil selon la session
`login/page.tsx`	Inscription et connexion
`(jeu)/layout.tsx`	Protection serveur des pages de jeu
`(jeu)/village/page.tsx`	Page qui monte l'interface Village

```
  api/[[...route]]/route.ts       Transmet /api/* à Hono
src/
  bootstrap/                      Assemblage de l'application et configuration serveur
  modules/
    realm/                        Domaine Village (API, application, domaine, stockage,
UI)
    system/                       Route de santé et composant d'état de l'API
  platform/
    auth/                         Contrat d'authentification et adaptateur Better Auth
    db/                           Connexion et sélection de la base
    event-store/                  Journal générique, schéma et reprise sur conflit
    http/                         Authentification des routes et validation JSON
  ui/                             Composants et présentation partagés
drizzle/                           Migrations SQL et métadonnées de schéma
docs/                              ADR, qualité et documentation
```

Frontières entre modules

Un module métier peut être utilisé depuis l'extérieur uniquement via ses entrées publiques :

- index.ts : fabrique et API côté serveur ;
- contracts.ts : schémas de validation et types partageables ;
- ui.tsx : interface destinée au navigateur.

Les fichiers internes d'un autre module ne sont pas importables. Le domaine métier ne dépend que de son propre dossier domain/, la couche application/ ne connaît pas les adaptateurs infrastructure/, et platform/ ignore les modules métier. dependency-cruiser, configuré dans .dependency-cruiser.cjs, vérifie ces règles avec pnpm lint:archi. Il détecte également les cycles et interdit d'embarquer du code serveur dans les composants client.

3. Parcours d'une requête

Affichage du jeu

1. Le navigateur demande /village. 2. Next.js exécute app/(jeu)/layout.tsx sur le serveur. 3. currentUser() lit les en-têtes de la requête et demande à Better Auth de résoudre la session. 4. Sans session, Next.js redirige vers /login avant de rendre le jeu. Avec une session, la page est affichée dans AppShell. 5. app/(jeu)/village/page.tsx monte VillageScreen, qui appartient au module realm.

Appel de l'API

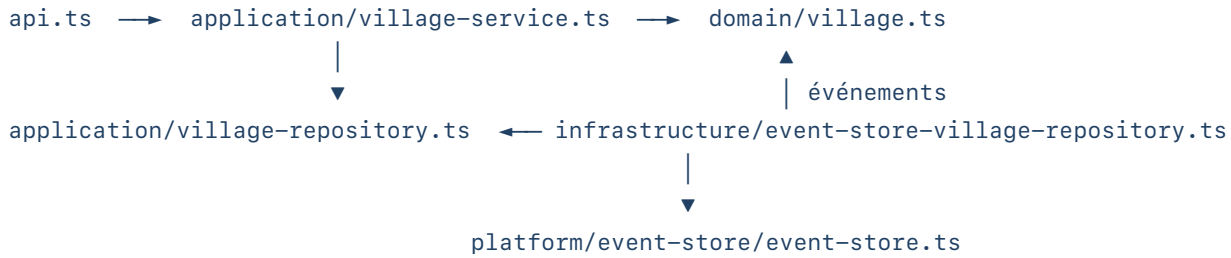
1. Le composant client appelle le client Hono typé à l'adresse /api/realm/.... 2. La route attrape-tout de Next.js délègue la requête à getServer().api.fetch(). 3. getServer() construit une seule instance serveur par processus et la conserve dans globalThis. La construction est paresseuse, pour ne pas ouvrir la base pendant l'analyse du projet par Next.js. 4. La racine de composition createApi() attend que la base soit prête, normalise les erreurs imprévues et monte les sous-applications. 5. Pour /realm/*, requireUser résout l'utilisateur. Une requête anonyme reçoit 401; l'identifiant authentifié est transmis à la route sous la forme userId. 6. La route du module valide le JSON avec Zod, appelle le service applicatif, puis traduit le résultat ou l'erreur métier en

réponse HTTP.

Le module system expose GET /api/system/health, sans authentification. Better Auth gère ses propres routes sous /api/auth/*.

4. Comment fonctionne le module Village ?

Le module applique une séparation inspirée de l'architecture hexagonale :



- Domaine (domain/) : état, commandes, événements, règles de production et coûts.

C'est du TypeScript pur ; il ne connaît ni HTTP, ni base de données, ni bibliothèque serveur.

- Application (application/) : orchestre le chargement, la décision métier,

l'enregistrement et le renvoi de la vue. Elle dépend du contrat VillageRepository, pas de son implémentation.

- Infrastructure (infrastructure/) : adapte le journal générique de la plateforme

au port du Village.

- API (api.ts) : valide les entrées, invoque le service et produit les réponses

HTTP. Les règles métier ne sont pas écrites dans les routes.

- Interface (ui.tsx) : présente le village, les ressources et les bâtiments, puis

envoie les commandes avec un client Hono typé.

Commandes, événements et règles du jeu

Une commande exprime une intention (FoundVillage, UpgradeBuilding). Le domaine vérifie si elle est permise et produit zéro ou plusieurs événements, faits accomplis immuables (VillageFounded, BuildingUpgraded). replay() repart de l'état vide et rejoue les événements pour reconstruire l'état courant : le village n'est pas stocké comme une ligne contenant son état complet.

À la fondation, le joueur reçoit 100 bois, 50 pierres et 0 or ; la scierie et la carrière démarrent au niveau 1, la mine au niveau 0. La production est calculée selon les niveaux : scierie 10 bois/minute/niveau, carrière 6 pierres/minute/niveau, mine 2 or/minute/niveau. Les quantités sont extrapolées depuis l'horodatage du dernier événement. L'interface rafraîchit son horloge deux fois par seconde et peut donc afficher le stock qui progresse sans requêter le serveur en continu.

Le coût d'une amélioration est calculé à partir du coût de base du catalogue multiplié par (niveau + 1)^1.3, puis arrondi. Le coût réellement payé et l'heure de l'action sont enregistrés dans l'événement. Cela préserve le sens des événements passés si les règles de coût ou de production changent ultérieurement.

Écritures simultanées

Chaque joueur possède un flux distinct, village-<userId>. Le journal impose une unicité sur (stream, version). Le service lit l'état et sa version, prend une décision, puis tente d'ajouter les événements attendus à la suite de cette version. Si un autre traitement a écrit entre-temps, la contrainte unique provoque un conflit ; le service recharge alors l'état récent et rejoue la commande, jusqu'à trois tentatives. Une autre erreur, ou un conflit persistant, n'est pas masqué.

Cette stratégie de concurrence optimiste est adaptée aux requêtes courtes ; elle évite les verrous explicites. Le journal est en ajout seul : il ne faut pas modifier les événements historiques en place.

5. Base de données et migrations

src/platform/db/client.ts choisit la base selon la configuration :

- Développement local : en l'absence de DATABASE_URL, PGlite stocke la base dans .data/pglite/. Les migrations du dossier drizzle/ sont appliquées au démarrage.
- Tests : PGlite en mémoire, initialisé avec les mêmes migrations et vidé entre les tests afin de préserver leur indépendance.
- Base hébergée : avec DATABASE_URL, le pilote pg ouvre un pool PostgreSQL. Le code attend que les migrations aient été appliquées dans le processus de livraison ; contrairement au mode local, elles ne sont pas exécutées au démarrage. Sous Vercel, l'absence d'URL est une erreur explicite, car le disque local y serait éphémère.

Drizzle décrit les schémas dans les fichiers schema.ts et drizzle-kit génère les migrations SQL. Deux schémas PostgreSQL sont actuellement utilisés :

- identity : tables user, session, account et verification nécessaires à Better Auth ; le compte possède une empreinte du mot de passe, pas le mot de passe en clair ;
- platform : table events, journal générique utilisé par le module Village.

La contrainte unique (stream, version) est aussi la garantie d'intégrité qui rend possible la détection des écritures concurrentes.

6. Connexion et sécurité

Better Auth gère l'inscription, la connexion, les sessions et la déconnexion. Le client React de connexion parle à /api/auth/* ; côté serveur, le même contrat AuthPort permet de remplacer l'adaptateur réel par un adaptateur simple en test. Le secret BETTER_AUTH_SECRET est obligatoire en production sur Vercel. En développement, une valeur réservée au local est utilisée. L'adresse publique peut être réglée avec BETTER_AUTH_URL ; par défaut, elle vaut http://localhost:3000.

La protection des pages du jeu se fait côté serveur ; la protection de l'API repose sur le middleware requireUser. Les requêtes de commandes sont validées par des schémas Zod. Les erreurs imprévues sont journalisées côté serveur puis renvoyées au client avec une réponse générique 500, sans détail technique.

Point d'attention connu : le registre de dette technique documente DT-004, encore ouverte : les compteurs de limitation des tentatives d'authentification sont conservés dans la mémoire de chaque instance, et ne sont donc pas partagés entre instances. Voir docs/qualite/dette-technique.md avant un déploiement multi-instance.

7. Dépendances et stack

Les versions ci-dessous viennent de package.json (les types Node/React sont spécifiés avec des plages de versions).

Exécution

- next 16.4.0 : routage, rendu serveur et intégration de l'application web.
- react / react-dom 19.3.0 : composants et interface.
- hono 4.13.13 : application et routes API modulaires.
- @hono/zod-validator 0.9.1 : validation des corps de requête avec Zod.
- zod 4.6.6 : schémas d'entrée et types déduits.
- better-auth 1.7.7 : comptes, authentification par mot de passe et sessions ;

better-auth/react fournit le client navigateur.

- drizzle-orm 0.45.4 : requêtes et schémas PostgreSQL typés.
- pg 8.23.1 : pilote et pool pour PostgreSQL externe.
- @electric-sql/pglite 0.5.9 : PostgreSQL embarqué pour le développement et les tests.

Développement et qualité

- typescript 6.0.3 : typage statique strict et vérification (pnpm typecheck).
- drizzle-kit 0.31.11 : génération des migrations (pnpm db:generate --name <nom>).
- vitest 5.0.4 : tests automatisés (pnpm test).
- @biomejs/biome 2.5.15 : formatage et analyse (pnpm lint).
- dependency-cruiser 18.6.0 : contrôle des frontières entre couches.
- @types/node, @types/pg, @types/react, @types/react-dom : types pour les

bibliothèques et l'environnement.

pnpm 10.34.7 est le gestionnaire de paquets déclaré. Node.js 22 ou supérieur est requis. Le dépôt contient pnpm-lock.yaml, qui verrouille la résolution effective des paquets.

8. Tests et commandes pratiques

Vitest exécute les tests serveur Node placés sous src//*.test.ts. Ils couvrent notamment les règles du domaine, le service, le journal, les routes, l'authentification et la composition. Les tests peuvent contrôler l'horloge, utilisent une base en mémoire et remplacent l'authentification par un adaptateur déterministe.

```
pnpm install
pnpm dev
```

L'application est disponible sur http://localhost:3000. Créer un compte depuis /login pour

accéder à /village.

- Commande: pnpm dev · Utilité: Démarrer le serveur de développement
- Commande: pnpm build puis pnpm start · Utilité: Construire puis lancer la version de production
- Commande: pnpm lint · Utilité: Vérifier formatage et règles Biome
- Commande: pnpm typecheck · Utilité: Vérifier les types TypeScript
- Commande: pnpm lint:archi · Utilité: Vérifier les frontières d'architecture
- Commande: pnpm test · Utilité: Exécuter les tests Vitest
- Commande: pnpm verify · Utilité: Enchaîner lint, types, architecture et tests
- Commande: pnpm db:generate --name <nom> · Utilité: Générer une migration Drizzle après un changement de schéma

Les variables prises en charge sont documentées dans .env.example. Pour une configuration locale personnalisée, copier ce fichier en .env.local ; ce dernier n'est pas versionné. La variable AUTH_RATE_LIMIT=off ne sert qu'à désactiver la limitation en local.

9. État actuel et limites à connaître

- Les modules réellement présents sont realm (le Village) et system (santé de l'API).

Les autres domaines évoqués dans l'ADR 0001 restent des évolutions possibles.

- Le projet est documenté comme non déployé à ce jour ; l'ADR évoque Vercel et Neon comme

cibles envisagées, tandis que la configuration du code permet une base PostgreSQL via DATABASE_URL.

- L'ADR 0002 sur l'Event Sourcing porte encore plusieurs mentions de contenu à compléter.

Il formalise le choix, mais le présent guide s'appuie sur l'implémentation pour décrire les détails et conséquences.

- En production, il faut configurer les secrets et appliquer les migrations sur la base

hébergée avant de servir l'application.

10. Ordre de lecture conseillé

Pour suivre le chemin d'une fonctionnalité, lire dans cet ordre :

1. app/(jeu)/village/page.tsx puis src/modules/realm/ui.tsx pour l'écran ; 2. src/modules/realm/api.ts et src/modules/realm/contracts.ts pour le contrat HTTP ; 3. src/modules/realm/application/village-service.ts pour l'orchestration ; 4. src/modules/realm/domain/village.ts et catalog.ts pour les règles métier ; 5. src/modules/realm/application/village-repository.ts pour le port ; 6. src/modules/realm/infrastructure/event-store-village-repository.ts pour l'adaptateur ; 7. src/platform/event-store/event-store.ts et src/platform/db/client.ts pour la persistance ; 8. src/bootstrap/create-api.ts pour comprendre comment toutes les pièces sont assemblées.

Les décisions structurantes sont consignées dans docs/adr/0001-monolithe-modulaire.md et docs/adr/0002-event-sourcing-village.md.

Pour régénérer la version PDF sur macOS après une mise à jour de ce guide :

```
swift scripts/render-architecture-pdf.swift docs/architecture-application.md
```